Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over $10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
CREATE TABLE BankCustomers (
   CustID NUMBER PRIMARY KEY,
   CustName VARCHAR2(50),
   CustAge NUMBER,
   AccountBalance NUMBER,
   VIPStatus VARCHAR2(5)
);
CREATE TABLE LoanDetails (
   LoanNo NUMBER PRIMARY KEY,
   CustID NUMBER,
   LoanInterest NUMBER,
   RepaymentDate DATE,
   FOREIGN KEY (CustID)
      REFERENCES BankCustomers(CustID)
);
INSERT INTO BankCustomers VALUES (1, 'Ravi', 65, 15000, 'N');
```

```
INSERT INTO BankCustomers VALUES (2, 'Neha', 45, 12000, 'N');
INSERT INTO BankCustomers VALUES (3, 'Kiran', 70, 8000, 'N');
INSERT INTO LoanDetails
VALUES (201, 1, 10, SYSDATE + 15);
INSERT INTO LoanDetails
VALUES (202, 2, 12, SYSDATE + 25);
INSERT INTO LoanDetails
VALUES (203, 3, 11, SYSDATE + 40);
COMMIT;
SELECT * FROM BankCustomers;
SELECT * FROM LoanDetails;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.158 seconds

	CUSTID	CUSTNAME	CUSTAGE	ACCOUNTBALANC	VIPSTATUS
1	1	Ravi	65	15000	N
2	2	Neha	45	12000	N
3	3	Kiran	70	8000	N

Scenario 1:

```
BEGIN
   FOR c IN (SELECT CustID, CustAge FROM BankCustomers)
   LOOP
      IF c.CustAge > 60 THEN
         UPDATE LoanDetails
         SET InterestRate = InterestRate - 1
         WHERE CustID = c.CustID;
      END IF;
   END LOOP;
   COMMIT;
   DBMS_OUTPUT.PUT_LINE('Interest rates updated successfully.');
END;
```

/

```
SQL> BEGIN
         FOR c IN (SELECT CustID, CustAge FROM BankCustomers)
         LOOP
             IF c.CustAge > 60 THEN...
Show more...


Interest rates updated successfully.


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.015
```

SELECT * FROM LoanDetails;

	LOANID	CUSTID	INTERESTRATE	DUEDATE
1	101	1	9	7/10/2026, 1:22:17
2	102	2	12	7/20/2026, 1:22:17
3	103	3	10	8/4/2026, 1:22:17 P

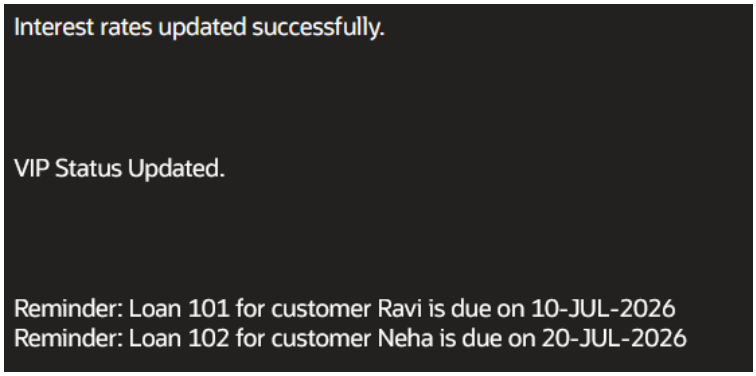
Scenario 2:

```
BEGIN
   FOR c IN (SELECT CustID, AccountBalance FROM BankCustomers)
   LOOP
      IF c.AccountBalance > 10000 THEN
         UPDATE BankCustomers
         SET VIPStatus = 'Y'
         WHERE CustID = c.CustID;
      END IF;
   END LOOP;
   COMMIT;
   DBMS_OUTPUT.PUT_LINE('VIP Status Updated.');
END;
/
SELECT * FROM BankCustomers;
```

	CUSTID	CUSTNAME	CUSTAGE	ACCOUNTBALANC	VIPSTATUS
1	1	Ravi	65	15000	Y
2	2	Neha	45	12000	Y
3	3	Kiran	70	8000	N

Scenario 3:

```
SET SERVEROUTPUT ON;
BEGIN
   FOR l IN (
      SELECT b.CustName,
           l.LoanID,
           l.DueDate
      FROM BankCustomers b
      JOIN LoanDetails l
      ON b.CustID = l.CustID
      WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30
   )
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Reminder: Loan ' || l.LoanID ||
         ' for customer ' || l.CustName ||
         ' is due on ' ||
         TO_CHAR(l.DueDate,'DD-MON-YYYY')
      );
   END LOOP;
END;
/
```



Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
```

```
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER
);
INSERT INTO Accounts VALUES (1, 101, 'SAVINGS', 10000);
INSERT INTO Accounts VALUES (2, 102, 'SAVINGS', 5000);
INSERT INTO Accounts VALUES (3, 103, 'CURRENT', 20000);
COMMIT;
SELECT * FROM Accounts;
```

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	1	101	SAVINGS	10000
2	2	102	SAVINGS	5000
3	3	103	CURRENT	20000

Scenario 1:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'SAVINGS';
    COMMIT;
END;
/
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'PROCESSMONTHLYINTEREST';
```

	OBJECT_NAME	OBJECT_TYPE
1	PROCESSMONTHLYI	PROCEDURE

```
BEGIN
    ProcessMonthlyInterest;
END;
/
```

```
SQL>     BEGIN
             ProcessMonthlyInterest;
     END;


PL/SQL procedure successfully completed.
```

```
SELECT * FROM Accounts;
```

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	1	101	SAVINGS	10100
2	2	102	SAVINGS	5050
3	3	103	CURRENT	20000

Scenario 2:

```
CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    EmployeeName VARCHAR2(50),
    DepartmentID NUMBER,
    Salary NUMBER
);
INSERT INTO Employees VALUES (1, 'Sai', 101, 50000);
INSERT INTO Employees VALUES (2, 'Abhi', 101, 60000);
INSERT INTO Employees VALUES (3, 'Ram', 102, 55000);
COMMIT;
SELECT * FROM Employees;
```

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	1	Sai	101	50000
2	2	Abhi	101	60000
3	3	Ram	102	55000

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
   p_DepartmentID IN NUMBER,
   p_BonusPercent IN NUMBER
)
AS
BEGIN
   UPDATE Employees
   SET Salary = Salary + (Salary * p_BonusPercent / 100)
   WHERE DepartmentID = p_DepartmentID;
   COMMIT;
END;
/
BEGIN
   UpdateEmployeeBonus(101, 10);
END;
/
```

```
SQL> BEGIN
        UpdateEmployeeBonus(101, 10);
     END;



PL/SQL procedure successfully completed.

Elapsed: 00:00:00.017
```

```
SELECT * FROM Employees;
```

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENTID	SALARY
1	1	Sai	101	55000
2	2	Abhi	101	66000
3	3	Ram	102	55000

Scenario 3:

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_FromAccount IN NUMBER,
    p_ToAccount IN NUMBER,
    p_Amount IN NUMBER
)
AS
    v_Balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_Balance
    FROM Accounts
    WHERE AccountID = p_FromAccount;
    IF v_Balance >= p_Amount THEN
        UPDATE Accounts
        SET Balance = Balance - p_Amount
        WHERE AccountID = p_FromAccount;
        UPDATE Accounts
        SET Balance = Balance + p_Amount
        WHERE AccountID = p_ToAccount;
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Transfer Successful');
```

```
    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Account Not Found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
BEGIN
    TransferFunds(1, 2, 2000);
END;
/
```

```
SQL> BEGIN
         TransferFunds(1, 2, 2000);
     END;


Transfer Successful


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.016
```

```
SELECT AccountID, Balance
FROM Accounts;
```

	ACCOUNTID	BALANCE
1	1	8100
2	2	7050
3	3	20000

```
BEGIN
    TransferFunds(1, 2, 50000);
```

END;

/

```
SQL> BEGIN
         TransferFunds(1, 2, 50000);
     END;


Insufficient Balance


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.005
```